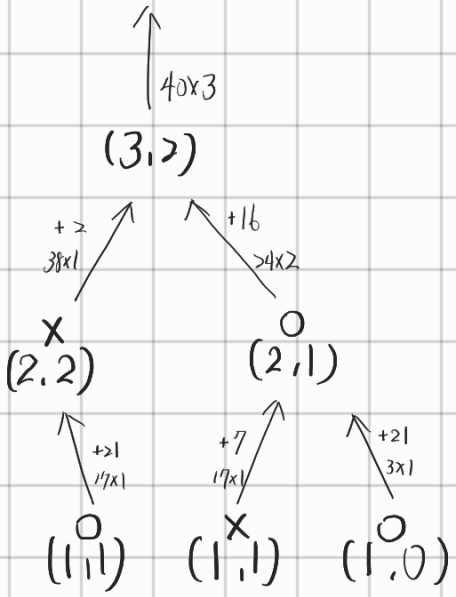
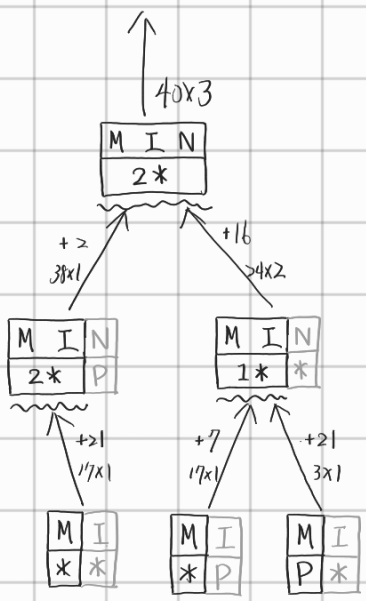
string1: 13 9 14 MIN

string2: 16 P





↗ 具有 return 的意思

O 為，該位配了 *

X 為，該位未配*，
配 string 2 当下殘餘的字母的最右那个

(3,2)：表示目前处理的情況是，
string 1 的 1~3 的字母，
搭配 2 个 *，还有 string 2 的 1 的字母。
找出在这情境下的最大值並 return.

(3,2) 的情境裡又可分為 2 种情況.

一个是，string 1 的第 3 个字母 P，不配 *，配 string 2 殘存的最右字母

$\boxed{\begin{matrix}MI\\2*\end{matrix}}$ @Max + $\boxed{\begin{matrix}N\\P\end{matrix}}$，也就是呼叫 (2,2)，得 return 的值後再加上 $\boxed{\begin{matrix}N\\P\end{matrix}}$ 值

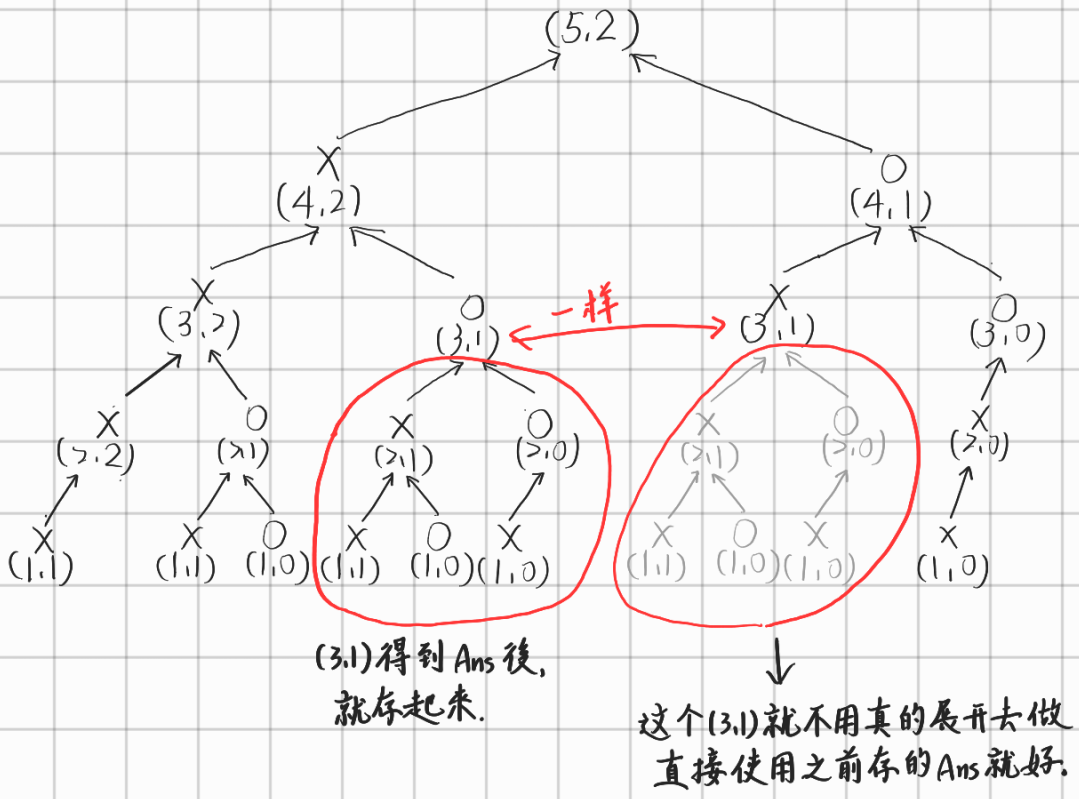
一个是，string 1 的第 3 个字母 P，配 *

$\boxed{\begin{matrix}MI\\1*\end{matrix}}$ @Max + $\boxed{\begin{matrix}N*\end{matrix}}$，也就是呼叫 (2,1)，得 return 的值後再加上 $\boxed{\begin{matrix}N*\end{matrix}}$ 值

完後，这 2 种情況的值相比較，較大的那个
，即為 (3,2) 的解，就可以結束屬于 (3,2) 的 function
，並 return 这个解

把递迴优化：

↳ 重複的東西，就不要再展开做
使用先前存起来的答案就可以了。



其实图中的(2,1)、(2,0)、(1,1)、(1,0)，也都有重複。作法同理
这些情况其实在做第一遍並把Ans存起来後。
後續再遇到，就不需要呼叫他們的function了
直接用Ans。

本体為假設舉例

```
#include<iostream>
#include<algorithm>
#include<string>
#include<map>
using namespace std;
long long s,mag;
int num;
map<int,map<int,long long> >my;
map<int,map<int,int> >mytime;

long long fun(string &s1,string &s2,int top1,int top2,int snum)
{
    long long r1=-1,r2=-1;
    if(top1==0)
    {
        if(top2==0)
        {
            my[0][0]=abs(s1[0]-s2[0]);
            mytime[0][0]=1;
            return abs(s1[0]-s2[0]);
        }
        else if(snum==1)
        {
            my[0][1]=abs(s1[0]-'A'+1-s);
            mytime[0][1]=1;
            return abs(s1[0]-'A'+1-s);
        }
    }

    if(top2>=0)
    {
        if(mytime[top1-1][snum]!=0)
            r1=my[top1-1][snum]+abs(s1[top1]-s2[top2]);
        else
            r1=fun(s1,s2,top1-1,top2-1,snum)+abs(s1[top1]-s2[top2]);

    }

    if(snum>=1)
    {
        if(mytime[top1-1][snum-1]!=0)
            r2=my[top1-1][snum-1]+abs(s1[top1]-'A'+1-s);
        else
            r2=fun(s1,s2,top1-1,top2,snum-1)+abs(s1[top1]-'A'+1-s);
    }

    if(r1>r2)
    {
        my[top1][snum]=r1;
        mytime[top1][snum]=mytime[top1-1][snum];
        return r1;
    }
    else if(r1<r2)
    {
```

目前str1最右边的index　目前str2最右边的index　目前所剩*的

(3-1)　(1-1)　2

假設当下為 MIN 2* 的function

(3,2)

(2,2)

(2,1)

MIN 2*　NP

MIN 1*　N*

my[top1][snum]=r1; //return Ans前,先把答案存好.

mytime[top1][snum]=mytime[top1-1][snum]; //儲存組合的,若勝者為(2,2),(3,2)的Ans的組合的即為(2,2)的Ans的組合的

return r1; //回傳(3,2)的Ans

```
        my[top1][snum]=r2;
        mytime[top1][snum]=mytime[top1-1][snum-1];//储存组合数,若胜者为(2,1),
                                                  (3,2)的Ans的组合数即为(2,1)的Ans的组合数
        return r2; //回传(3,2)的Ans
    }
    else  //r1=r2,两种情况Ans的值相同时,组合数mytime[3][2]要累加
    {
        my[top1][snum]=r1;
        mytime[top1][snum]=mytime[top1-1][snum]+mytime[top1-1][snum-1];
        return r1;//回传(3,2)的Ans
                                  (2,2)组合数          (2,1)组合数
    }
}

int main()
{
    string s1,s2;
    while(cin>>s1)
    {
        mag=-1;
        num=0;
        my.clear();
        mytime.clear();
        cin>>s2;
        cin>>s;
        int s1len=s1.length();
        int s2len=s2.length();
        if(s1len>=s2len)//较长的string做主.
        {
                          s1的index      s2的index      *数
            mag=fun(s1,s2,s1len-1,s2len-1,s1len-s2len);
            num=mytime[s1len-1][s1len-s2len];
        }
        else
        {
            mag=fun(s2,s1,s2len-1,s1len-1,s2len-s1len);
            num=mytime[s2len-1][s2len-s1len];
        }

        cout<<mag<<" "<<num<<endl;
    }
    return 0;
}
```